Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 1 - Containerize a sample application using Docker

Question 1: Containerize a sample application using Docker: write a Dockerfile, build the image, and run the container. Document the image layers and container behavior.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64) with Docker Engine 29.9.3. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Application

`app/app.py` is a small Flask service served by gunicorn. It exposes `/` (service info, owner and the container hostname that served the request), `/health` (status and version) and `/visits` (a Redis-backed counter used in Q2). Three pytest unit tests cover the endpoints (`tests/test_app.py`, 3 passed, see `docs/logs/q0-retry.log`).

Dockerfile

```
FROM public.ecr.aws/docker/library/python:3.12-slim
ARG APP_VERSION=2.0.0
ENV APP_VERSION=${APP_VERSION} PYTHONUNBUFFERED=1
WORKDIR /srv
# Dependencies before source: code edits keep this layer cached.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
RUN useradd --uid 10001 --no-create-home web
USER web
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=3s CMD python -c "import urllib.request as u; u.urlopen('h
ttp://127.0.0.1:5000/health')"
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Design choices:

- `python:3.12-slim` from the AWS public mirror of Docker Official Images (the same image as Docker Hub's `python:3.12-slim`, without Hub's anonymous pull limits).
- Requirements are copied and installed before the source, so code edits do not reinstall dependencies.
- The process runs as an unprivileged user (uid 10001), not root.
- `HEALTHCHECK` lets Docker report whether the app actually answers, not just whether the process is alive.
- `APP_VERSION` is a build argument, reused later to build 2.0.1 for the rolling update (Q6) and per-build versions in CD (Q7).

Build

```
$ docker build -f docker/Dockerfile -t ensp461-lab2:2.0.0 . --progress=plain
#5 [1/6] FROM public.ecr.aws/docker/library/python:3.12-slim@sha256:4840d5...    DONE 90.3s
#6 [2/6] WORKDIR /srv                                                           DONE 0.1s
#7 [3/6] COPY app/requirements.txt .                                            DONE 0.1s
#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt                     DONE 14.1s
#9 [5/6] COPY app/ .                                                            DONE 0.1s
#10 [6/6] RUN useradd --uid 10001 --no-create-home web                          DONE 0.2s
```

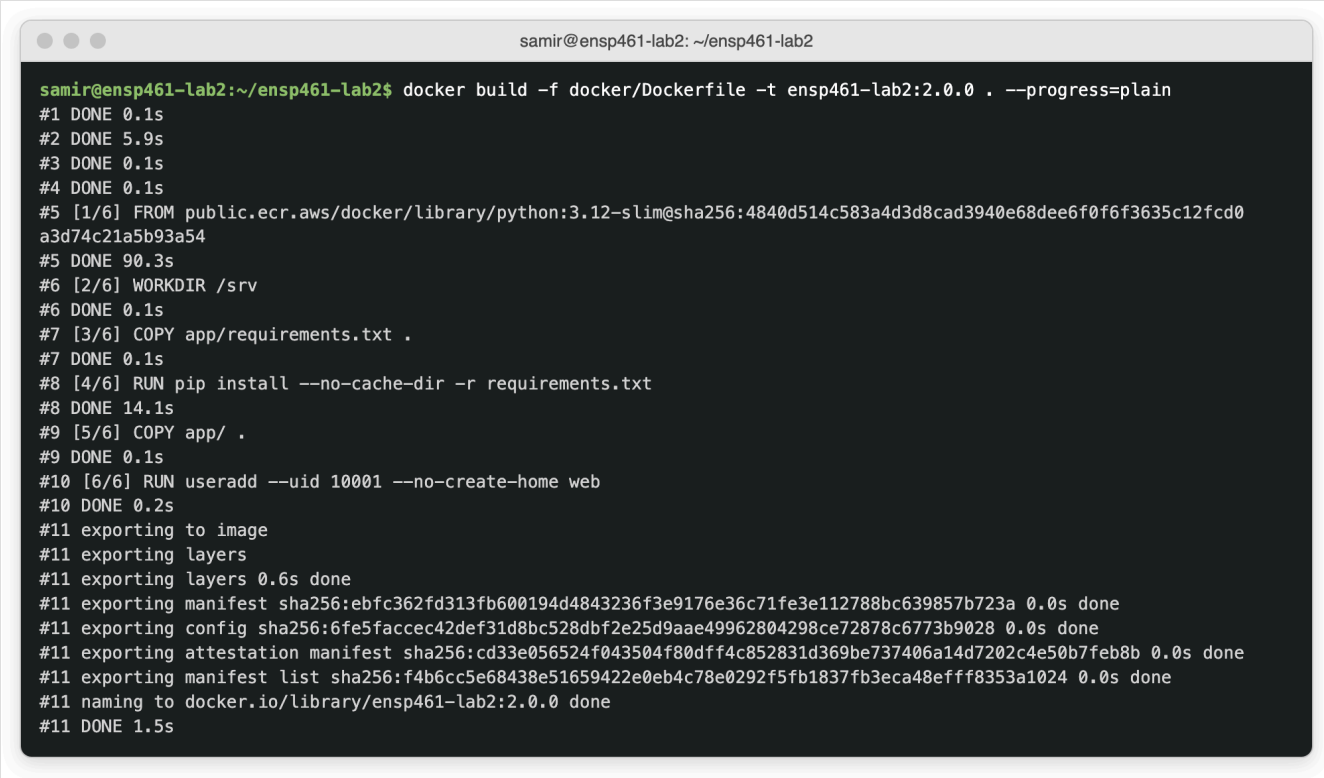


Fig: Image build with BuildKit plain progress

Image layers

```
$ docker images ensp461-lab2
IMAGE                ID             DISK USAGE   CONTENT SIZE
ensp461-lab2:2.0.0   f4b6cc5e6843        227MB         49.9MB
$ docker image inspect ensp461-lab2:2.0.0 --format '{{len .RootFS.Layers}} layers'
9 layers
```

Layer (top to bottom of `docker history`)	Size	Source
CMD, HEALTHCHECK, EXPOSE, USER	0 B	metadata only
`RUN useradd ... web`	41 kB	this Dockerfile
`COPY app/ .`	4.1 kB	this Dockerfile
`RUN pip install ...`	19.7 MB	this Dockerfile
`COPY app/requirements.txt .`	4.1 kB	this Dockerfile
`WORKDIR /srv`, ARG, ENV	0 B	this Dockerfile
Python 3.12.16 install, apt packages, Debian trixie rootfs	43.7 MB + 5.6 MB + 109 MB	base image

The image has 9 filesystem layers: 4 from the base image and 5 from this Dockerfile. Instructions such as `ENV`, `EXPOSE`, `USER` and `CMD` change only image metadata and add 0 B layers in `docker history`. Almost all of the size is the base image; the app adds about 20 MB, nearly all of it the pip install.

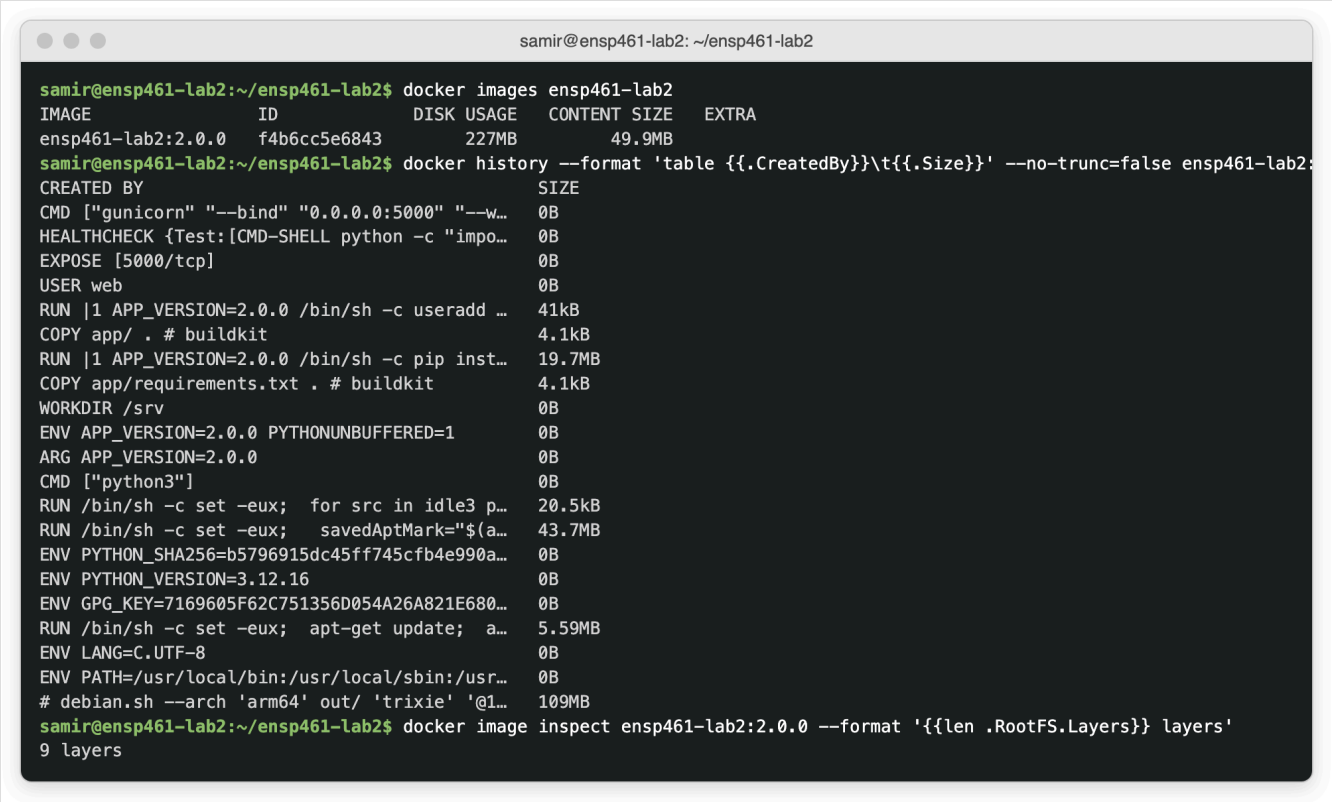


Fig: Image size, history and layer count

Run and container behavior

```
$ docker run -d --name lab2-web -p 5000:5000 ensp461-lab2:2.0.0
$ curl -s localhost:5000/
{"environment":"local","owner":"Samir Kumar Gupta","served_by":"b0c1b704db82","service":"ensp46
1-lab2","version":"2.0.0"}
$ curl -s localhost:5000/health
{"status":"UP","version":"2.0.0"}
$ docker exec lab2-web id
uid=10001(web) gid=10001(web) groups=10001(web)
$ docker inspect lab2-web --format 'health={{.State.Health.Status}}'
health=healthy
```

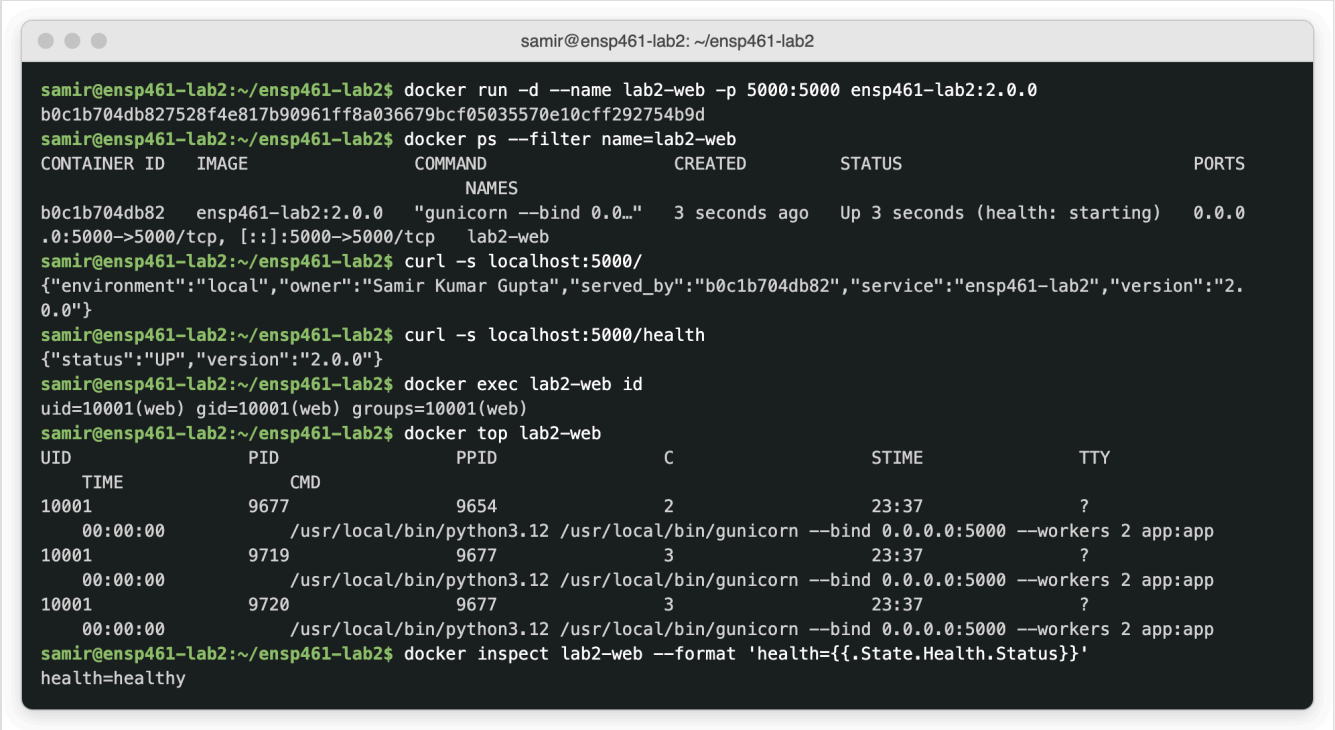


Fig: Container run, HTTP responses, user and processes

Observed behavior:

- `served_by` equals the container ID: the container has its own hostname (UTS namespace).
- `docker top` shows one gunicorn master and two workers, all as uid 10001, confirming the non-root `USER`.
- Status moved from `health: starting` to `healthy` once the first health check passed.
- The port mapping `0.0.0.0:5000->5000/tcp` publishes the container port on the host.

Layer cache

Rebuilding after `touch app/app.py` reused every layer (`CACHED` for steps 6 to 10). BuildKit keys `COPY` layers on file contents, not modification time, so a timestamp-only change does not invalidate the cache. A real code edit only rebuilds from `COPY app/ .` onward; the 14 s dependency layer stays cached because of the instruction order.

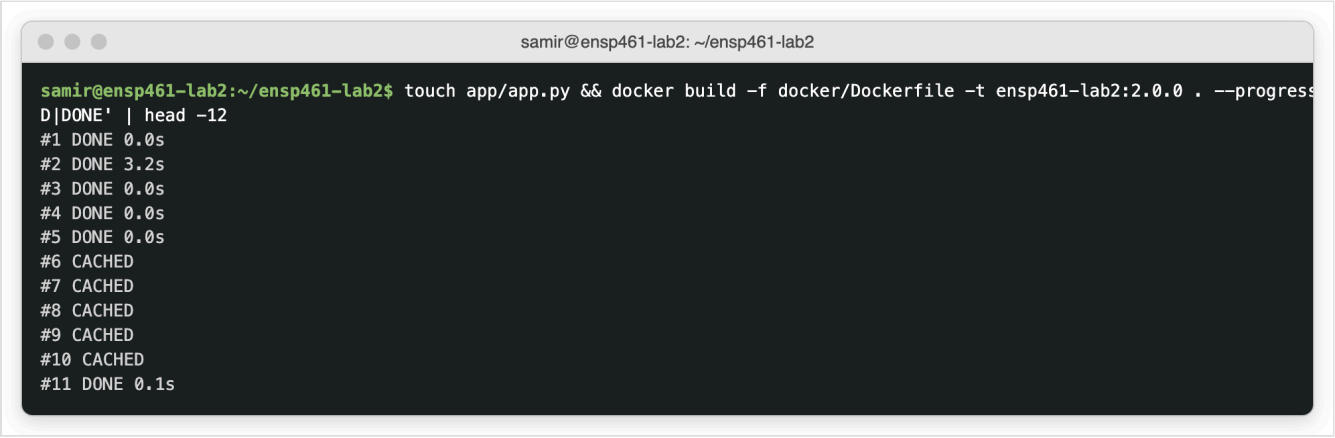


Fig: Rebuild after touch: all layers cached